Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Python Programming for Finance

TGS Ref No: TGS-2026065049

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	23 August 2026	Initial CASL/WSQ release aligned to the seven course topics, 17 realistic finance labs and current assessment design.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

How to Use This Guide **4**

Before You Start — What You Need 4

The Evidence Standard 4

Course Learning Outcomes **4**

Topic 01 — Introduction to Python Programming **4**

Key Concepts 4

Lab 01 — Set Up a Reproducible Finance Project 5

Lab 02 — Translate a Finance Requirement into an SGX Price Tool 7

Topic 02 — Data Types and Operators **8**

Key Concepts 8

Lab 03 — Build a Loan Amortisation Calculator 8

Lab 04 — Model a Portfolio with Python Collections 10

Topic 03 — Problem Solving with Control Structures **11**

Key Concepts 11

Lab 05 — Encode PE and ROE Valuation Rules 11

Lab 06 — Loop Through a Portfolio and Build Comprehensions 13

Topic 04 — Scripting with Functions and Lambda **14**

Key Concepts 14

Lab 07 — Create Reusable Financial Ratio Functions 14

Lab 08 — Screen Customers with Lambda, Map and Filter 16

Topic 05 — Import and Process Finance Data **17**

Key Concepts 17

Lab 09 — Import and Clean a Finance DataFrame 17

Lab 10 — Filter and Slice Finance Data Safely 19

Lab 11 — Run a Finance Data-Quality Audit 20

Topic 06 — Aggregate and Visualize Finance Data **21**

Key Concepts 21

Lab 12 — Join, Group and Pivot a Loan Portfolio 21

Lab 13 — Visualize Daily Returns and Distribution 23

Lab 14 — Build a Moving-Average Technical Analysis 24

Topic 07 — Analyze Finance Data **25**

Key Concepts 25

Lab 15 — Build a Clean-to-Risk Pipeline with pipe 25

Lab 16 — Analyse Statistics, Correlation and Rolling Risk 27

Lab 17 — Capstone: Produce an Executive Finance Analytics Pack 28

Quick Command Reference **29**

Assessment Flow **29**

Support **29**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

How to Use This Guide

This guide supports Python Programming for Finance (TGS-2026065049). It contains the detailed, step-by-step procedure for all 17 hands-on labs. The slide deck is deliberately concept-led; use this guide and the matching lab folder when you need exact commands, files, evidence and acceptance criteria.

Before You Start — What You Need

- A Windows or macOS laptop with internet access and permission to install Python tools.
- Python 3.11 or later; Google Colab is an acceptable browser-based alternative for demonstrations.
- Visual Studio Code, JupyterLab or another Python IDE.
- uv or venv for isolated environments; Pandas, NumPy and Matplotlib for the analytical labs.
- The complete labs/ folder, including the supplied offline CSV data. Live market-data refresh is optional.
- Never use confidential client data in class; use supplied mock data or approved anonymised data only.

The Evidence Standard

Execution	The script or notebook runs without an unhandled traceback.
Validation	Tests, row counts, totals or known values support correctness.
Artifact	The required CSV, chart, log or report is saved in the lab folder.
Interpretation	One sentence explains what the output means for the finance scenario.

Course Learning Outcomes

- LO1: Translate finance business requirements into clear Python programming objectives and configure a reproducible development environment.
- LO2: Use Python data types and operators to represent, calculate and present financial information accurately.
- LO3: Organise finance logic with conditional statements, loops and comprehensions to solve repeatable analytical problems.
- LO4: Create reusable functions and lambda-based transformations for finance business use cases.
- LO5: Import, inspect, filter, slice and clean finance data with Pandas Series and DataFrames.
- LO6: Join, aggregate, test and visualise finance data using groupby, pivot tables and appropriate charts.
- LO7: Improve analytical code with apply and pipe, use descriptive statistics, and analyse changes in financial data over time.

Topic 01 — Introduction to Python Programming

Business requirements · finance use cases · environment · first script

Key Concepts

- Business first: Define the decision, users, data, output and acceptance criteria before writing code.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Finance fit: Python connects market data, calculations, visualisation, automation and AI in one ecosystem.
- Reproducibility: A locked environment makes analytical results easier to review, repeat and audit.
- Accountability: The analyst remains responsible for validating inputs, formulas, assumptions and outputs.

Lab 01 — Set Up a Reproducible Finance Project

Alignment	LO1 / A1
Scenario	A bank analytics team needs every learner to run the same package versions and verify the environment before analysing SGX data.
Goal	A verified Python project with an environment check and dependency record.
Tools	Python 3, uv or venv, Pandas, Matplotlib
Folder	labs/lab-01-…

Step-by-step

1. Create the lab project folder and open a terminal in it.

```
uv init python-finance-lab01
```

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Add the analysis packages.

```
uv add pandas numpy matplotlib yfinance
```

3. Create env_check.py from the supplied starter file.

```
cp starter.py env_check.py
```

4. Run the environment check.

```
uv run python env_check.py
```

5. Record the Python and package versions in evidence.txt.

```
uv tree > evidence.txt
```

6. Review the output for missing packages or tracebacks.

Acceptance Criteria

The script prints the Python, Pandas, NumPy and Matplotlib versions and ends with ENVIRONMENT READY.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 02 — Translate a Finance Requirement into an SGX Price Tool

Alignment	LO1
Scenario	A wealth desk wants a repeatable daily price summary for a small basket of Singapore-listed equities.
Goal	A requirement canvas, acceptance tests and an offline-safe SGX price summary script.
Tools	Python, CSV, optional yfinance
Folder	labs/lab-02-…

Step-by-step

1. Read scenario.md and identify users, inputs, rules, outputs and risks.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Complete requirement_canvas.md before coding.

3. Run the starter against the supplied fallback prices.

```
python starter.py
```

4. Implement load_prices() and format_report() in solution.py.

5. Add a test for the invalid ticker row.

```
python -m unittest test_solution.py
```

6. Save the output as evidence.txt.

```
python solution.py > evidence.txt
```

Acceptance Criteria

The report prints five labelled SGX prices to two decimals, handles an invalid ticker without crashing, and all tests pass.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 02 — Data Types and Operators

Numbers · strings · lists · tuples · dictionaries · sets · operators

Key Concepts

- Numbers: Use int for counts and float or Decimal for measured financial values.
- Strings: Clean labels, identifiers, dates and report text before matching or presenting them.
- Collections: Lists, tuples, dictionaries and sets organise positions, identifiers and attributes.
- Operators: Arithmetic, comparison and logical operators encode calculations and decisions.

Lab 03 — Build a Loan Amortisation Calculator

Alignment	LO2
Scenario	A relationship manager needs a transparent monthly-instalment calculation including the zero-interest staff-loan case.
Goal	A loan calculator with monthly payment, total repayment and total interest.
Tools	Python, math, csv
Folder	labs/lab-03-…

Step-by-step

1. Open scenario.md and note the principal, annual rate and term.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Run the starter and observe the incomplete result.

```
python starter.py
```

3. Implement the amortisation formula in monthly_payment().

4. Handle a zero annual rate without division by zero.

5. Generate the first 12 schedule rows.

```
python solution.py
```

6. Run the unit tests and save the output.

```
python -m unittest test_solution.py
```

Acceptance Criteria

For $500,000 at 2.6% over 25 years, payment is about $2,268.35; a 0% loan returns principal divided by months.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 04 — Model a Portfolio with Python Collections

Alignment	LO2
Scenario	An investment team needs a clean in-memory representation of holdings, identifiers, sector groups and prices.
Goal	A portfolio report using list, tuple, dictionary and set operations.
Tools	Python, csv
Folder	labs/lab-04-…

Step-by-step

1. Inspect portfolio.csv and identify suitable Python types.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Load rows into a list of dictionaries.

```
python starter.py
```

3. Represent the fixed instrument key as a tuple.

4. Build ticker-to-price and ticker-to-sector dictionaries.

5. Use sets to calculate overlap between two portfolios.

6. Print market value and weights; confirm weights total 100%.

```
python solution.py
```

Acceptance Criteria

Every holding has a market value, portfolio weights sum to 100%, and the overlap set is reported without duplicates.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 03 — Problem Solving with Control Structures

If/elif/else · loops · range · enumerate · zip · comprehensions

Key Concepts

- Branching: Use if/elif/else when a finance rule selects one outcome from alternatives.
- Iteration: Use loops to apply the same logic to positions, periods or scenarios.
- Loop control: break stops a search; continue skips an invalid record without ending the run.
- Comprehensions: Create a transformed collection concisely when the rule remains readable.

Lab 05 — Encode PE and ROE Valuation Rules

Alignment	LO3
Scenario	An equity screen must classify securities consistently at defined PE and ROE thresholds.
Goal	A boundary-tested BUY/HOLD/REVIEW valuation classifier.
Tools	Python, unittest
Folder	labs/lab-05-…

Step-by-step

1. Read the valuation policy and underline every boundary.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Run the starter with PE values 14.99, 15, 25 and 25.01.

```
python starter.py
```

3. Implement classify_valuation() with explicit inclusive boundaries.

4. Combine PE and ROE conditions into the final recommendation.

5. Run boundary tests.

```
python -m unittest test_solution.py
```

6. Explain one classification in evidence.txt.

```
python solution.py > evidence.txt
```

Acceptance Criteria

All exact-boundary tests pass and each recommendation includes the rule that produced it.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 06 — Loop Through a Portfolio and Build Comprehensions

Alignment	LO3
Scenario	A portfolio analyst needs valuations for every valid position while malformed records are logged and excluded.
Goal	A reconciled holdings table, exception log and concise list/dict comprehensions.
Tools	Python, csv
Folder	labs/lab-06-…

Step-by-step

1. Inspect holdings.csv and find the deliberately malformed row.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Iterate with enumerate() to number records.

3. Use continue to skip and log the malformed holding.

4. Accumulate total value and verify it against the output rows.

```
python solution.py
```

5. Create a list comprehension for overweight positions.

6. Create a dictionary comprehension for ticker-to-weight mapping.

```
python -m unittest test_solution.py
```

Acceptance Criteria

The invalid row is logged once, valid positions reconcile to the portfolio total, and comprehension results match the loop results.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 04 — Scripting with Functions and Lambda

Functions · parameters · return values · scope · lambda · map · filter

Key Concepts

- Single purpose: A function should perform one understandable analytical responsibility.
- Inputs and outputs: Parameters make assumptions visible; return values keep computation separate from display.
- Reuse: Well-named functions reduce repeated formulas and inconsistent fixes.
- Lambda: Use small anonymous functions for short transformations, not complex business rules.

Lab 07 — Create Reusable Financial Ratio Functions

Alignment	LO4
Scenario	A bank reporting team calculates margin, ROE and ROA in several reports and needs one tested implementation.
Goal	A ratios module returning named results with validation and tests.
Tools	Python, dataclasses, unittest
Folder	labs/lab-07-…

Step-by-step

1. Review the required function contract and units.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Implement safe_divide() with a defensible zero-denominator result.

3. Implement net_margin(), roe() and roa().

4. Return a dictionary of named ratios from calculate_ratios().

```
python solution.py
```

5. Test typical, zero and negative cases.

```
python -m unittest test_solution.py
```

6. Document assumptions in evidence.txt.

Acceptance Criteria

The sample produces margin about 55.95%, ROE about 18.09% and ROA about 1.53%; tests pass.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 08 — Screen Customers with Lambda, Map and Filter

Alignment	LO4
Scenario	A retail bank wants to transform customer income into affordability ratios and filter applications for manual review.
Goal	A readable transformation pipeline plus an equivalent named-function implementation.
Tools	Python, csv, unittest
Folder	labs/lab-08-…

Step-by-step

1. Load applicants.csv and count the input records.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Use map with a lambda to add monthly_income.

3. Use filter to select high debt-to-income records.

```
python starter.py
```

4. Rewrite the risk rule as a named function.

5. Confirm both versions return identical IDs.

```
python -m unittest test_solution.py
```

6. Explain which implementation is easier to audit.

Acceptance Criteria

Lambda and named-function pipelines select the same applicant IDs, and counts reconcile to the input.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 05 — Import and Process Finance Data

Pandas · Series · DataFrame · import · filter · slice · clean

Key Concepts

- Series: A labelled one-dimensional array, such as closing prices indexed by date.
- DataFrame: A labelled table of rows and columns suitable for finance records and time series.
- Selection: Use labels and conditions deliberately with loc, iloc and boolean masks.
- Data quality: Missing, duplicate, invalid and inconsistent values must be measured before analysis.

Lab 09 — Import and Clean a Finance DataFrame

Alignment	LO5
Scenario	A treasury extract contains duplicated dates, missing prices and inconsistent date/value types.
Goal	A curated price DataFrame and a written data-quality reconciliation.
Tools	Pandas, CSV
Folder	labs/lab-09-…

Step-by-step

1. Load dirty_prices.csv with Pandas and inspect head, info and dtypes.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Parse Date and coerce numeric price columns.

3. Measure duplicates and missing values before changing data.

```
python starter.py
```

4. Remove duplicate dates using the latest record.

5. Fill only short Close-price gaps and document the assumption.

6. Save clean_prices.csv and quality_report.txt.

```
python solution.py
```

Acceptance Criteria

The curated file has unique sorted dates, numeric prices, no missing Close values, and a reconciliation from raw to clean rows.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 10 — Filter and Slice Finance Data Safely

Alignment	LO5
Scenario	A portfolio manager asks for bank-sector positions above a materiality threshold and a date-windowed price series.
Goal	A filtered positions file and labelled versus positional slice comparison.
Tools	Pandas, CSV
Folder	labs/lab-10-…

Step-by-step

1. Load portfolio.csv and market_prices.csv.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Use loc with a boolean mask for Bank sector and Value above threshold.

3. Use loc for an inclusive date-label slice.

4. Use iloc for a positional sample and compare endpoints.

```
python starter.py
```

5. Sort without mutating the original DataFrame.

6. Save filtered_positions.csv and run tests.

```
python -m unittest test_solution.py
```

Acceptance Criteria

The filter returns only qualifying bank positions; loc and iloc row counts match their documented boundary rules.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 11 — Run a Finance Data-Quality Audit

Alignment	LO5
Scenario	A credit portfolio extract must pass completeness, uniqueness, range and reconciliation controls before modelling.
Goal	A reusable audit function and exception report for loan records.
Tools	Pandas, CSV, JSON
Folder	labs/lab-11-…

Step-by-step

1. Load loans_dirty.csv and define the audit controls.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Check required columns and row uniqueness.

3. Measure missingness by column.

4. Test ranges for income, debt-to-income and credit score.

```
python starter.py
```

5. Write exceptions.csv and audit_summary.json.

```
python solution.py
```

6. Confirm exception counts reconcile to the summary.

```
python -m unittest test_solution.py
```

Acceptance Criteria

The audit identifies every planted defect, produces machine-readable counts and leaves the raw file unchanged.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 06 — Aggregate and Visualize Finance Data

concat · merge · groupby · pivot table · returns · charts

Key Concepts

- Combine: concat stacks compatible data; merge joins tables using explicit keys.
- Aggregate: groupby splits records, applies calculations and combines results.
- Reshape: Pivot tables summarise a measure across two or more dimensions.
- Visualise: Choose a chart that matches the analytical question and label units clearly.

Lab 12 — Join, Group and Pivot a Loan Portfolio

Alignment	LO6
Scenario	A risk team needs exposure and approval metrics by sector, region and risk band from separate application and reference tables.
Goal	A validated merge, groupby summary and pivot table exported to CSV.
Tools	Pandas, CSV
Folder	labs/lab-12-…

Step-by-step

1. Profile application and sector reference keys.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Merge with validate='many_to_one' and indicator=True.

3. Review unmatched rows before dropping or fixing them.

```
python starter.py
```

4. Group by sector and calculate count, exposure and mean credit score.

5. Create a region-by-risk-band exposure pivot.

```
python solution.py
```

6. Reconcile grouped and pivot totals to the merged portfolio.

```
python -m unittest test_solution.py
```

Acceptance Criteria

No unexpected many-to-many join occurs, unmatched keys are reported, and summary exposure totals equal the source.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 13 — Visualize Daily Returns and Distribution

Alignment	LO6 / A4 / A5
Scenario	An investment committee wants to understand daily price changes and the distribution of returns for DBS.
Goal	A return series, line chart and histogram with labelled analytical takeaways.
Tools	Pandas, Matplotlib, CSV
Folder	labs/lab-13-…

Step-by-step

1. Load dbs_prices.csv and sort by Date.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Calculate Close.pct_change() as Daily_Return.

3. Drop the first expected NaN from chart input.

```
python starter.py
```

4. Plot daily returns through time with a zero line.

5. Plot a histogram with mean and tail markers.

```
python solution.py
```

6. Record mean, standard deviation and largest absolute return.

Acceptance Criteria

returns.csv has one fewer non-null row than prices; charts render; summary statistics match Pandas calculations.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 14 — Build a Moving-Average Technical Analysis

Alignment	LO6 / A2–A3 / A6–A9
Scenario	A quantitative analyst needs an OHLC view and 10/50-day moving averages with objectively detected crossover dates.
Goal	An offline-safe technical-analysis report with OHLC, moving averages and crossover evidence.
Tools	Pandas, Matplotlib, optional yfinance
Folder	labs/lab-14-…

Step-by-step

1. Load the supplied DBS OHLC data; optionally refresh with yfinance.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Validate required OHLC columns and date ordering.

3. Calculate MA10 and MA50 with rolling().mean().

```
python starter.py
```

4. Detect sign changes in MA10 minus MA50.

5. Plot Close, MA10 and MA50 with crossover markers.

```
python solution.py
```

6. Save crossover_dates.csv and explain the first crossover.

Acceptance Criteria

The output contains OHLC rows, MA10/MA50 values, at least one crossover date, and a labelled chart; the offline file always works.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 07 — Analyze Finance Data

apply · pipe · statistics · correlation · rolling analysis · change tracking

Key Concepts

- apply: Run a function across rows or columns when vectorised operations are not practical.
- pipe: Express a readable sequence of named DataFrame transformations.
- Statistics: Describe centre, spread, relationships and unusual observations before drawing conclusions.
- Change over time: Returns, rolling windows and drawdown expose behaviour that levels alone can hide.

Lab 15 — Build a Clean-to-Risk Pipeline with pipe

Alignment	LO7
Scenario	A credit-risk team wants the DataFrame transformations to read like an auditable sequence rather than one large notebook cell.
Goal	A chain of named clean, enrich, score and summarise functions connected with pipe().
Tools	Pandas, CSV, unittest
Folder	labs/lab-15-…

Step-by-step

1. Load loan_pipeline.csv and preserve a raw copy for comparison.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Write clean_data(df) without mutating its input.
3. Write add_features(df) for affordability and utilisation ratios.
4. Write score_risk(df) using vectorised conditions or apply.

```
python starter.py
```

5. Chain the functions with pipe and export results.

```
python solution.py
```

6. Run tests for immutability, boundaries and row counts.

```
python -m unittest test_solution.py
```

Acceptance Criteria

Each stage can be tested alone, the raw DataFrame is unchanged, and the final risk-band counts reconcile to total rows.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 16 — Analyse Statistics, Correlation and Rolling Risk

Alignment	LO7
Scenario	A portfolio risk analyst must profile returns, compare bank-stock relationships and track changing volatility over time.
Goal	A statistics table, correlation matrix, rolling-volatility series and drawdown chart.
Tools	Pandas, NumPy, Matplotlib, CSV
Folder	labs/lab-16-…

Step-by-step

1. Load multi_asset_prices.csv and calculate daily returns.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Compute count, mean, median, standard deviation and quantiles.

3. Calculate covariance and correlation matrices.

```
python starter.py
```

4. Calculate 20-day rolling volatility and annualise using sqrt(252).

5. Calculate cumulative return and drawdown.

```
python solution.py
```

6. Check correlation symmetry and drawdown sign conventions.

```
python -m unittest test_solution.py
```

Acceptance Criteria

Correlation is symmetric with 1 on the diagonal, rolling volatility has the expected warm-up NaNs, and maximum drawdown is non-positive.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 17 — Capstone: Produce an Executive Finance Analytics Pack

Alignment	LO1–LO7
Scenario	The CFO requests a concise, reproducible evidence pack combining data quality, portfolio exposure, returns, risk and recommendations.
Goal	A self-contained HTML report, charts, curated CSVs, validation log and executive summary.
Tools	Python, Pandas, Matplotlib, HTML
Folder	labs/lab-17-…

Step-by-step

1. Review the acceptance checklist and supplied portfolio data.

> ***Expected evidence: the correct lab folder and source files are visible before any calculation is run.***

2. Run the data-quality audit and save the reconciliation.
3. Calculate position values, weights and sector exposure.
4. Calculate return, volatility and drawdown indicators.

```
python starter.py
```

5. Generate charts and the HTML executive report.

```
python solution.py
```

6. Run validate_outputs.py and capture PASS evidence.

```
python validate_outputs.py
```

Acceptance Criteria

All required files exist, totals reconcile, charts have labels, the HTML report opens locally, and validate_outputs.py prints CAPSTONE PASS.

Evidence Checklist

- Executed output or test result
- Generated CSV/chart/report requested in the goal
- Row-count or total reconciliation where applicable
- One-sentence finance interpretation

Troubleshooting

- Run the command from the individual lab folder so relative data paths resolve.
- Check the expected column names, data types, date order, currency and units.
- Treat the supplied offline data as the baseline; optional live data may produce different values.
- If a result looks plausible but fails a reconciliation, stop and inspect intermediate rows before presenting it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Quick Command Reference

Command	Purpose
python solution.py	Run the reference implementation from the current lab folder.
python -m unittest test_solution.py	Run the lab's automated checks.
uv run python solution.py	Run with the locked uv environment.
python -m pip install -r requirements.txt	Install the lab dependencies with pip.
jupyter lab	Open notebooks and files in JupyterLab.
python validate_outputs.py	Validate the capstone evidence pack.

Assessment Flow

1. Scan the TRAQOM assessment digital-attendance QR code.
2. Complete the Written Assessment (WA) individually.
3. Complete the Practical Performance (PP) individually.
4. Upload the two completed question papers on https://lms-tms.tertiaryinfotech.com/
5. Sign the Assessment Summary Record.

Support

Courseware and assessment portal: https://lms-tms.tertiaryinfotech.com/
Email: enquiry@tertiaryinfotech.com
Telephone: +65 6100 0613
Website: www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg